Food Order Management System

Using a Doubly Linked List in C++

1. Problem Statement

Design and implement a **Food Order Management System** in C++ using a **Doubly Linked List** that allows a user to place, view, search and cancel food orders from a predefined menu, and to calculate the total bill.

In a restaurant or canteen, orders are constantly added and removed. A fixed-size array handles this poorly because it wastes memory and makes deletion from the middle costly. A doubly linked list suits this need: nodes are created dynamically as orders arrive, any order can be removed by updating its links, and the list can be traversed in both directions.

2. System Description

- **Menu:** A fixed menu of 6 items (Pizza, Burger, Sandwich, French Fries, Dosa, Cold Coffee) with their prices is stored in arrays.
- **Order record:** Each order is stored as a node containing an auto-generated Order ID, food item, quantity, amount, and *prev* / *next* pointers.
- **List pointers:** Global *head* and *tail* pointers track the first and last orders.
- **Interface:** A menu-driven console program that repeats until the user exits.

No.	Food Item	Price (Rs.)
1	Pizza	150
2	Burger	80
3	Sandwich	60
4	French Fries	70
5	Dosa	50
6	Cold Coffee	90

3. Operations to be Implemented

1. **Add order:** Insert a new node at the end of the list, with validation of the food number (1-6) and quantity (greater than 0).
2. **Display all orders:** Traverse forward from head and show the total bill.
3. **Display in reverse:** Traverse backward from tail.
4. **Search order:** Find an order by Order ID.
5. **Cancel order:** Delete a node by Order ID, updating the prev/next links, head and tail.
6. **Calculate total bill:** Sum the amounts of all orders.
7. **Display menu:** Show the food list with prices.

8 **Exit:** Free all dynamically allocated memory.

4. Objectives

Main objective: To apply the concepts of dynamic memory allocation, structures, and doubly linked list operations (insertion, deletion, forward and backward traversal, searching) in a menu-driven real-life application.

Specific objectives:

- To understand and use **structures** and **pointers** to build a node with data and two links.
- To use **dynamic memory allocation** (*new* / *delete*) to create and remove nodes at run time.
- To perform **insertion at the end** of a doubly linked list in O(1) time using the tail pointer.
- To perform **deletion** of a node from the head, middle or tail by correctly updating the prev/next links and the head and tail pointers.
- To implement **forward and backward traversal** and **linear search** on a list.
- To validate user input so that invalid food numbers and quantities are rejected.
- To prevent **memory leaks** by releasing all nodes before the program exits.
- To develop a modular, **menu-driven program** that solves a practical problem.

5. Expected Outcome

A working console application in which a user can add orders from the menu, view them in forward or reverse order, search or cancel any order by its ID, and get the correct total bill at any time, with all memory released on exit.